

Text indexing and Boolean query model

Hoa Dinh Nguyen, PhD.

hoand@ptit.edu.vn

Lecture Overview: Text Indexing and Boolean Retrieval

- ▶ Text preprocessing: tokenization, stop-word removal, stemming, synonym reduction
- ▶ Term weighting: TF, DF, TF-IDF; importance of terms within and across documents
- ▶ Index structures: flat files, signature files, inverted files (standard IR index)
- ▶ Boolean query model: AND, OR, NOT; positional constraints; weighted Boolean
- ▶ Goal: understand how unstructured text becomes searchable in a multimedia database



Text Preprocessing Pipeline

- ▶ Tokenization: split raw text into words/tokens (handle punctuation, numbers, hyphenation)
- ▶ Case folding: convert to lowercase so 'Database' and 'database' map to the same term
- ▶ Stop-word removal: discard high-frequency function words (the, is, at) with little discriminative power
- ▶ Stemming: reduce inflected forms to a root (Porter stemmer: retrieval, retrieved → retriev)
- ▶ Lemma/synonym normalization: map synonyms to one representative term (car, automobile → car)
- ▶ Output: a bag of index terms (keywords) per document, ready for weighting and indexing



Keyword extraction for Text

- ▶ Collect of all the words contained in the text document
- ▶ Remove stop-words

Excerpt of Common Stop Words

A	ALTHOUGH	ANYONE
ABOUT	ALWAYS	ANYTHING
ACROSS	AMONG	ANYWHERE
AFTER	AMONGST	ARE
AFTERWARDS	AN	AROUND
AGAIN	AND	AS
AGAINST	ANOTHER	AT
ALL	ANY	BE
ALSO	ANYHOW	BECOME

Keyword extraction for Text

- ▶ Reduction: Words with the same meaning or common root words are replaced by a single word (Root Words)
 - ▶ Example: retrieval, retrieved, retrieving, retrieve -> retrieve
- ▶ Find synonyms, choose a representative (wildcard) word.
 - ▶ Example: study, learning, schoolwork, reading,... -> study
- ▶ Weighting keywords.

Exercise — Preprocessing Practice

- ▶ Given the sentence:
 - ▶ 'The researchers are studying advanced database retrieval algorithms for multimedia systems.'
- ▶ Tasks:
 - ▶ (a) Tokenize and apply case folding
 - ▶ (b) Remove English stop-words; list remaining tokens
 - ▶ (c) Apply stemming manually (or with Porter): list stemmed forms
 - ▶ (d) Group synonyms: {study, learning, research} → study; rewrite the term list
- ▶ Compare how many distinct index terms remain after each step



Keyword extraction for Text

- ▶ How to weigh keywords

- ▶ Term Frequency (TF): tf_{ij} = count of term j in document i (often log-normalized)
- ▶ Document Frequency (DF): df_j = number of documents containing term j
- ▶ Inverse Document Frequency: $IDF = \log\left(\frac{N}{df_j}\right)$, rare terms receive higher weight
- ▶ TF-IDF:

$$W_{ij} = tf_{ij} \times \log\left(\frac{N}{df_j}\right)$$

TF-IDF Weighting — Worked Intuition

- ▶ Example: $N = 1000$ docs; 'database' in 50 docs $\rightarrow$ $IDF = \log(1000/50) \approx 3.0$
 - ▶ If $tf = 4$ in doc $i \rightarrow W_{ij} \approx 12.0$ (important term in that document)
- ▶ Rare, frequent-in-doc terms score highest; common words (after stop-word removal) score low



Indexing and Retrieval systems

- ▶ Indexing: Find the most characteristic features of the text to store and search for information.
- ▶ Difference between DBMS and Indexing and Retrieval systems

DBMS	IR Systems
Uniformly structured records, Include a fixed set of attributes to clearly and generally present the content of each record.	Records do not have common specific structures, no fixed attributes. The text is indexed based on keywords, descriptive information, markups, etc. used to express the content in a certain aspect.

Indexing and Retrieval systems

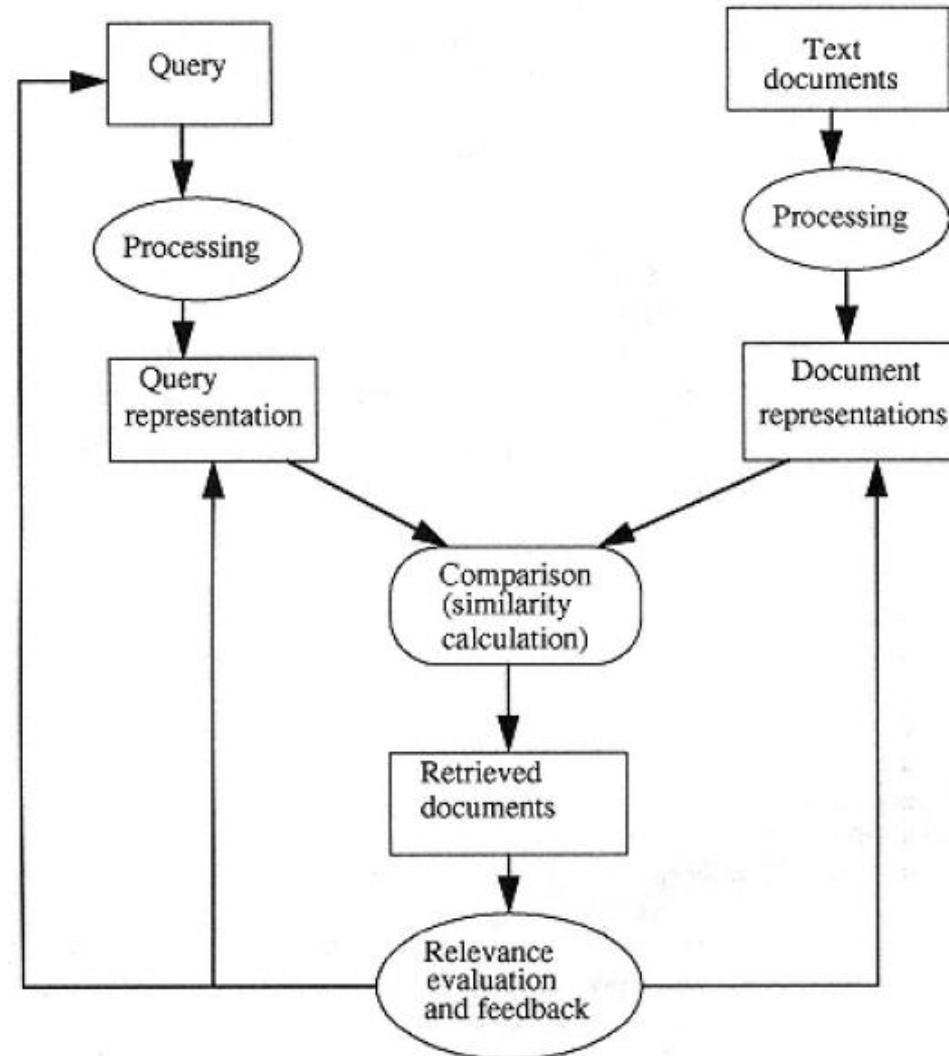
DBMS	IR Systems
Search data based on data matching on each attribute	Data searching is based on content similarities between query and data records
The search result must be exactly matched to the query	The search results are not necessarily exactly matched to the content of the query

Information Retrieval vs Database Queries

- ▶ DBMS: exact match on structured attributes (SELECT * WHERE year = 2024 AND dept = 'CS')
- ▶ IR: best-effort content match — user wants relevant documents, not necessarily exact wording
- ▶ Precision: fraction of retrieved documents that are relevant = $|\text{relevant} \cap \text{retrieved}| / |\text{retrieved}|$
- ▶ Recall: fraction of all relevant documents that were retrieved = $|\text{relevant} \cap \text{retrieved}| / |\text{relevant}|$
- ▶ Boolean model: high precision when query is carefully crafted; recall may suffer (AND too restrictive)
- ▶ Trade-off: broad OR queries increase recall but decrease precision



Indexing and Retrieval systems



Architecture of a Text Indexing and Retrieval System

- ▶ Offline indexing phase: collect documents → preprocess → build inverted index → store on disk
- ▶ Online query phase: parse query → lookup postings lists → merge/intersect → return doc IDs
- ▶ Components: document parser, lexicon (vocabulary), postings file, query processor
- ▶ Optional: query expansion (synonyms), spell correction, result ranking (extended Boolean / vector model)
- ▶ In multimedia DB: text index links to document OIDs; media metadata stored separately

Indexing and Retrieval systems with Boolean operator

▶ Data File Structure

▶ Flat files:

- ▶ One file contains one or more records
- ▶ Non-indexed text
- ▶ Search through text pattern matching

▶ Signature files:

- ▶ Contains features (string of bits) of information of text records
- ▶ There are many ways to extract features for each text
- ▶ The query also includes the features of the same kind

Signature Files — Theory and Trade-offs

- ▶ Idea: hash each document's term set into a fixed-length bit signature (superimposed coding)
- ▶ Query signature: OR the bit patterns of query terms; match if $(query_{sig} \text{ AND } doc_{sig}) = query_{sig}$
- ▶ Advantage: compact; fast sequential scan over signatures before accessing full text
- ▶ Disadvantage: false positives — different terms may hash to the same bit (collision)
- ▶ Use case: small collections, append-only archives, or as a first filter before inverted index
- ▶ Contrast: inverted files give exact term-to-document mapping with no false positives



Indexing and Retrieval systems with Boolean operator

- ▶ Inverted files: is the most common structure
 - ▶ Each keyword (term) is assigned a distinctive index
 - ▶ Each text (record) is assigned an identifier (ID)
 - ▶ Each keyword is used to store all the records that contain it
 - ▶ Records containing the same keyword are saved in a row corresponding to that keyword
 - ▶ Example
 - Term 1: Record 1, Record 3
 - Term 2: Record 1, Record 2
 - Term 3: Record 2, Record 3, Record 4
 - Term 4: Record 1, Record 2, Record 3, Record 4

Inverted Index — Structure and Construction

- ▶ Lexicon (dictionary): sorted list of all unique terms in the collection
- ▶ Postings list: for each term, sorted list of document IDs (and optionally positions/weights)
- ▶ Construction (offline): for each doc $\rightarrow$ extract terms $\rightarrow$ for each term, append docID to postings
- ▶ Merge step: sort terms; merge duplicate entries; compress postings (gap encoding, variable-byte)
- ▶ Query lookup: $O(1)$ dictionary lookup + $O(|\text{postings}|)$ scan for each query term
- ▶ AND query: intersect two sorted postings lists (merge algorithm, linear in list sizes)



Indexing and Retrieval systems with Boolean operator

- ▶ Boolean Query System

- ▶ Texts are indexed by keyword sets
- ▶ Queries consist of keywords accompanied by Boolean operators
 - ▶ AND (example: term 1 AND term 2)
 - ▶ OR (example: term 1 OR term 2)
 - ▶ NOT (example: term 1 AND NOT term 2) is often used to limit search results.

Indexing and Retrieval systems with Boolean operator

- ▶ Inverted files: use the Boolean model to query data
 - ▶ AND: (Term 1 AND Term 3) returns Record 3 as the result
 - ▶ OR: (Term 1 OR Term 2) returns Record 1, Record 2, Record 3 as the results
 - ▶ NOT: (Term 4 AND NOT Term 1) returns Record 2, Record 4 as the results
- ❖ 2 issues to pay attention to when searching by keywords:
 - ▶ Placement of keywords in text,
 - ▶ The importance of keywords to the content of the text

Boolean Query Processing on Inverted Files

- ▶ Query tree: (Term1 AND Term2) OR Term3 → evaluate inner nodes first (postings intersection/union)
- ▶ AND: intersect sorted postings lists → only docs containing ALL terms
- ▶ OR: union sorted postings lists → docs containing ANY term
- ▶ NOT: complement relative to full collection (requires knowing N and all doc IDs)
- ▶ Optimization: process AND from smallest postings list first (reduces intermediate result size)
- ▶ Limitation: no relevance ranking in pure Boolean model (unless weights added)



Indexing and Retrieval systems with Boolean operator

- ▶ The information about the location of the keywords needs to be added to the inverted file.
 - ▶ **Term *i*: Record no., Paragraph no., Sentence no., Word no.**
 - ▶ Example:
 - ▶ *world*: R99, 10, 8, 3; R155, 15, 3, 6; R166, 2, 3, 1
 - ▶ *battle*: R77, 9, 7, 2; R99, 10, 8, 6; R166, 10, 2, 5; R166, 2, 3, 2
 - ▶ Query: Find records where **“world” and “battle” are in the same sentence.**
- ▶ Information about keyword spacing
 - ▶ “In the same sentence”
 - ▶ Term 1 in the same sentence as Term 2
 - ▶ “Adjacent”
 - ▶ Term 1 adjacent to Term 2

Positional Indexing and Proximity Queries

- ▶ Extended inverted entry: $\text{term} \rightarrow (\text{docID}, \text{paragraph}, \text{sentence}, \text{wordPosition})^+$
- ▶ Phrase query 'world war': find positions of 'world' and 'war' where wordPosition differs by 1
- ▶ Same-sentence constraint: paragraph AND sentence must match for both terms
- ▶ Adjacent (within k words): $|\text{pos}(\text{term1}) - \text{pos}(\text{term2})| \leq k$
- ▶ Applications: legal search, biomedical literature, named entity proximity
- ▶ Storage cost: positional index is larger than document-level index but enables precise matching



Indexing and Retrieval systems with Boolean operator

- ▶ Boolean queries with weighted keywords
 - ▶ OR: If a record has multiple query keywords, the highest weight is selected. The records are arranged in weighted order from high to low.
 - ▶ **Term1**: R1, 0.3; R3, 0.5; R6, 0.8; R7, 0.2; R11, 0.1
 - ▶ **Term2**: R2, 0.7; R3, 0.6; R7, 0.5; R9, 0.5
 - ▶ **Term3**: R1, 0.8; R2, 0.4; R9, 0.7
 - ▶ Query: **Term 2 OR Term 3** return results R1, R2, R9, R3, R7
 - ▶ AND: Lowest weight will be selected for records with multiple query keywords
 - ▶ Query: **Term 2 AND Term 3** Return Results R9, R2
 - ▶ NOT: The difference between the weights of records with the same keyword will be used
 - ▶ Query: **Term 2 AND NOT Term 3** Return Results R3, R7, R2.

Extended Boolean Model and Limitations

- ▶ Weighted Boolean: OR $\rightarrow$ max weight; AND $\rightarrow$ min weight; NOT $\rightarrow$ weight difference
- ▶ Results can be ranked by computed weight — bridges Boolean and ranked retrieval
- ▶ Pure Boolean limitations:
 - ▶ Difficult for non-expert users to formulate effective queries
 - ▶ Equal treatment of all matched docs (no ranking in basic model)
 - ▶ AND-heavy queries often return empty or tiny result sets
- ▶ Modern systems: Boolean for filters + vector/BM25 ranking for ordering (hybrid retrieval)



Indexing and Retrieval systems with Boolean operator

- ▶ Summary of automatic indexing steps
 - ▶ Find all the words in the title, summary, text body
 - ▶ Remove the stop words
 - ▶ Find synonyms, replace them with a generic word
 - ▶ Find the root word of words with the same root
 - ▶ Find appearance frequencies of each keyword in the text
 - ▶ Find the weight of each keyword
 - ▶ Create an inverted file.

Summary — Text Indexing and Boolean Retrieval

- ▶ Preprocessing transforms raw text into a normalized term vocabulary
- ▶ TF-IDF weights terms by importance within a document and rarity across the collection
- ▶ Inverted files are the standard index for Boolean and ranked text retrieval
- ▶ Boolean operators (AND/OR/NOT) operate on postings lists; positional extensions add precision
- ▶ Weighted Boolean enables basic ranking; modern systems combine Boolean filters with BM25/vector ranking
- ▶ Next topics: vector space model, relevance feedback, and full-text search in multimedia DBMS

